CHƯƠNG 2 · BÀI 1

2.1. Tổng quan về DOM

Tài liệu học tập JavaScript Từ Zero Đến Hero · Nội dung biên soạn trong dự án StudyOnline.

> **Mục tiêu bài học**
>
> - Hiểu DOM là gì và mô hình cây phần tử
> - Chọn phần tử bằng querySelector và các phương thức khác
> - Thay đổi nội dung, thuộc tính và kiểu của phần tử

1. DOM là gì

DOM (Document Object Model) là biểu diễn cây của trang HTML. Mỗi thẻ là một nút trong cây; JavaScript dùng đối tượng `document` để đọc và thay đổi trang mà không cần tải lại.

2. Chọn phần tử

Để chọn một phần tử theo id, dùng `document.getElementById("myId")`. Các phương thức khác: `querySelector` (trả về phần tử đầu tiên khớp selector CSS), `querySelectorAll` (trả về danh sách mọi phần tử khớp) và `getElementsByClassName`.

```
const tieuDe = document.querySelector("h1");
const cacMuc = document.querySelectorAll(".item");
```

3. Thay đổi nội dung và kiểu

Có thể thay đổi nội dung bằng thuộc tính `textContent` (chỉ văn bản, an toàn) hoặc `innerHTML` (chèn HTML, cần cẩn thận với dữ liệu người dùng vì có nguy cơ tấn công XSS). Thuộc tính `style` và `classList` dùng để đổi giao diện.

```
tieuDe.textContent = "Xin chào DOM";
tieuDe.style.color = "tomato";
tieuDe.classList.add("noi-bat");
```

4. Tạo và xoá phần tử

`document.createElement("li")` tạo phần tử mới; `cha.appendChild(con)` thêm vào cây; `phanTu.remove()` xoá khỏi cây.

```
const li = document.createElement("li");
li.textContent = "Mục mới";
document.querySelector("ul").appendChild(li);
```

Tóm tắt

- DOM là cây phần tử của trang, truy cập qua `document`.
- Dùng `querySelector` / `querySelectorAll` để chọn phần tử.
- `textContent` an toàn hơn `innerHTML` khi hiển thị dữ liệu người dùng.

Câu hỏi ôn tập và bài tập vận dụng

1. Đổi nội dung và màu của một tiêu đề khi trang tải xong.
2. Tạo danh sách 5 mục bằng vòng lặp và createElement.
3. Giải thích vì sao không nên gán dữ liệu người dùng vào innerHTML.

Tổng quan về DOM — Thực hành và hỏi đáp

Phần bổ sung cho bài học · biên soạn 29/10/2026

Tình huống, ví dụ và tự kiểm tra

Hỏi thực tế: "getElementById('btn') ra null nên nút không hoạt động." Kiểm tra phần tử có đúng id không, có được render không và script có chạy trước khi HTML được phân tích không. Không kết luận một nguyên nhân khi chưa có HTML và vị trí script.

```
const button = document.getElementById("btn");
if (!button) {
  console.error("Không tìm thấy #btn; kiểm tra HTML và thời điểm chạy");
} else {
  button.textContent = "Sẵn sàng";
}
```

`textContent` phù hợp để hiển thị văn bản người dùng nhập. Không ghép dữ liệu chưa tin cậy vào innerHTML. Dùng defer cho script ngoài hoặc gắn sau DOMContentLoaded khi cần.

Tự luyện: đổi id trong HTML rồi quan sát thông báo. **Hướng giải:** sau khi khớp id và thời điểm, đối tượng không còn null; tránh thêm setTimeout tùy ý để che lỗi thứ tự.

Nguồn học tiếp

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide